

A QUANT RISK ENGINE · IN 16 MIB · NO OPERATING
SYSTEM

**We priced a derivative
and measured its risk on
a computer with no OS –
and checked it against
the exact answer.**

Monte Carlo, hardware randomness, and a Black–
Scholes ground truth.

This is the market Return Infinity sells into:
deterministic, auditable compute for finance.

THE PROBLEM

Two questions a trading desk asks every day.

What is this option worth? — and how much could I lose?

Real portfolios have no pricing formula, and risk is not one number but a whole loss distribution. The only general tool is simulation: draw thousands of random futures, price each, and read the answer off the spread.

ONE UNIKERNEL, THE WHOLE PIPELINE

Every external service, load-bearing.

Telegram – the instrument, in plain English

Firecrawl – live market data over HTTPS

Gemini LLM – parse it into parameters

RDRAND – the random price paths

compute – simulate, price, measure risk

Redis – an immutable audit record

PNG + Telegram – the chart and the answer back

THE VALIDATION — THE PART THAT MAKES IT REAL

Simulation, meet the exact answer.

A European call has a closed form: Black–Scholes. If the simulation lands on it within its error bar, the engine is correct.

METHOD	CALL PRICE
Black–Scholes (exact)	10.4506
Monte Carlo, 8M paths	10.4521 ± 0.0026
delta (both)	0.6368

Within one standard error. The random walk found the analytic price.

A QUANT TRICK, SHOWN HONESTLY

4x

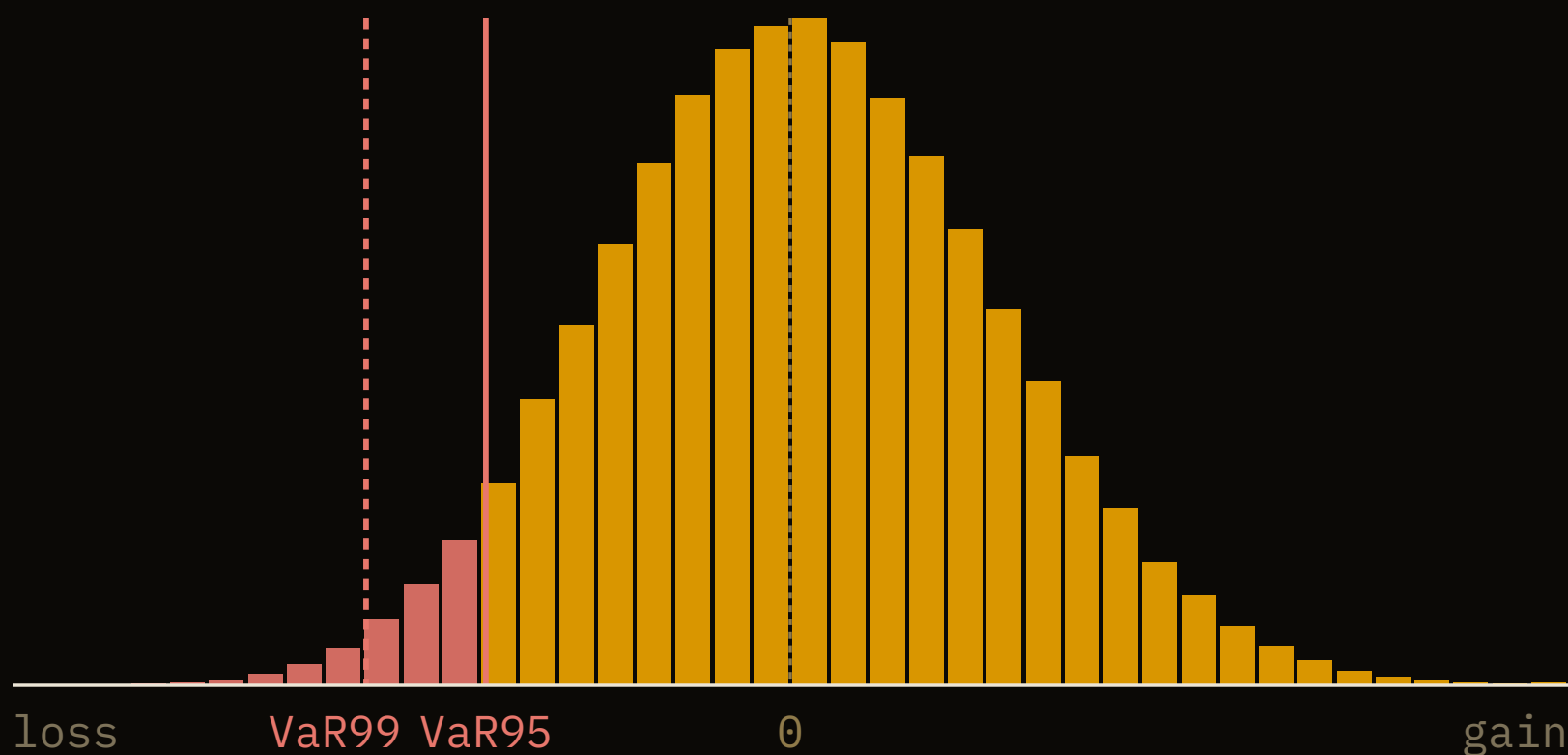
less variance, for free.

Pair every random shock Z with its mirror $-Z$ — antithetic variates. The two cancel each other's noise, so the error bar halves without a single extra sample.

plain	10.4486	+ -	0.0052	
antithetic	10.4521	+ -	0.0026	(4.0x)

THE RISK, FROM THE LOSS DISTRIBUTION

10-day Value-at-Risk on a \$1M position.



VaR95 = \$62,372 VaR99 = \$86,914 ES99 = \$98,985

WHY A UNIKERNEL, AND NOT A CONTAINER

Every run is logged, and every run is reproducible.

Each simulation writes an immutable record to Redis — the audit trail MiFID II and SEC rules require. And reproducibility is not free: this same platform silently computed wrong **1 in 4,000** times until we root-caused and fixed it.

A risk number you cannot reproduce is a risk number you cannot defend.

Determinism is the product. We found the one bug that broke it, and closed it.

WHERE THE RANDOMNESS CAME FROM

The CPU's own entropy, proven clean.

Millions of Gaussian shocks, each built from **RDRAND** — the hardware source shown the day before to pass ent, FIPS 140-2 and dieharder. A unikernel has no **/dev/random**; this one makes its own, correctly.

No math library either: exp, ln, sqrt and the normal distribution are a few dozen lines of C. The program *is* the machine.

OPEN SOURCE

The engine, the RNG, and the proofs.

github.com/tabibazar/unikernel-c

Monte Carlo pricing with variance reduction and Greeks, Value-at-Risk from the simulated distribution, the hardware RNG it runs on, and the evidence for all of it — in 16 MiB, with no operating system.

Built on Return Infinity's BareMetal. Prices by Black-Scholes; the randomness is the machine's own.